SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Department of Artificial Intelligence and Machine Learning

PROJECT REPORT

VisionTrack AI : A Real-Time Computer Vision Pipeline for Motion Analysis and Object Detection

Student Name	Sanjoli Gupta
Registration No.	23BAI11387
Subject	Computer Vision
Faculty	Mr. Santosh Kumar
Programme	B.Tech – Artificial Intelligence
Academic Year	2024 – 2025
Date of Submission	26 March 2026

Submitted for requirements of the course

Computer Vision (CSE4099)

VIT Bhopal University, Kothri Kalan, Sehore – 466114, Madhya Pradesh

1. Problem Statement

The question at the centre of this project is deceptively simple to ask but genuinely difficult to answer: given an ordinary video recording, can a computer reliably identify what is in the scene, which objects are moving, and where the camera itself is heading — all at the same time, and fast enough to matter in the real world? Each part of that question is a substantial research area on its own. Putting them together in a way that actually works on standard hardware, without exotic sensors or a data centre in the background, is where the engineering challenge begins.

Object detection has come a long way since the early days of sliding-window classifiers. Modern detectors can identify dozens of categories from a single image with impressive accuracy. But knowing that something is a car is quite different from knowing whether that car is moving or stationary, and knowing neither tells you anything about where you, the observer, are going. Background subtraction addresses the motion question but does so blindly — it sees foreground pixels, not objects, and it has no idea whether the moving region is a person, a bicycle, or a windblown plastic bag. Visual odometry, meanwhile, works on the geometry of the scene and can reconstruct a camera trajectory, but on its own it is agnostic about what the camera is looking at.

The problem this project tackles is therefore one of integration: bringing these three strands together so that the output of each informs and enriches the others. A detected car labelled as moving is a fundamentally more useful piece of information than either a detected car or a moving region alone. A trajectory estimated in the same pass as object detection begins to sketch a coherent picture of a dynamic scene — exactly the kind of picture that a driver assistance system or a delivery robot would need to act on.

2. Motivation and Relevance

It is worth pausing to consider why this particular combination of problems matters, rather than treating any one of them as sufficient on its own.

Take the autonomous vehicle context first. A self-driving car perceives the world through cameras and, increasingly, lidar. It must not only detect a pedestrian in its path but also predict whether that pedestrian is about to step forward or has already stopped at the kerb. The Moving/Static label that this pipeline generates is, at a conceptual level, exactly the kind of

annotation a downstream planning module would consume. Without it, a detected object is just an obstacle to be avoided; with it, the system can begin to reason about trajectories and intent.

Surveillance is another domain where the combination pays off. A camera watching a car park will generate thousands of false positives if it simply fires an alert every time a background subtractor detects foreground pixels — leaves blow, shadows shift, lighting changes. Filtering those alerts through an object detector that insists the foreground region must contain a recognisable object of interest reduces noise dramatically. Combining that filter with the Moving/Static heuristic further reduces it: a car that has been sitting in the same spot for two hours is technically part of the foreground if the background model has not yet adapted, but the motion label will correctly flag it as static.

For the study of computer vision as a discipline, the project is valuable for a different reason. It covers, in one working implementation, the three pillars that underpin the majority of real-world CV systems: geometric vision (the Essential Matrix and camera pose), statistical image processing (MOG2 background subtraction), and learned feature recognition (MobileNet SSD). Understanding how these three families of techniques relate to each other — where they agree, where they diverge, and where one compensates for the weaknesses of another — is more instructive than studying any of them in isolation.

3. System Design and Approach

The pipeline is structured as a sequence of loosely coupled processing stages, each encapsulated in its own Python module. Frames flow in from a video file, pass through four parallel analysis branches, and converge in a fusion step that produces the final annotated output. The decision to keep the stages modular was deliberate: it makes the system easier to debug, easier to extend, and easier to replace any single component without disturbing the rest.

3.1 System Architecture

The diagram below captures the full data flow, from the raw video source through the four processing branches to the display outputs.

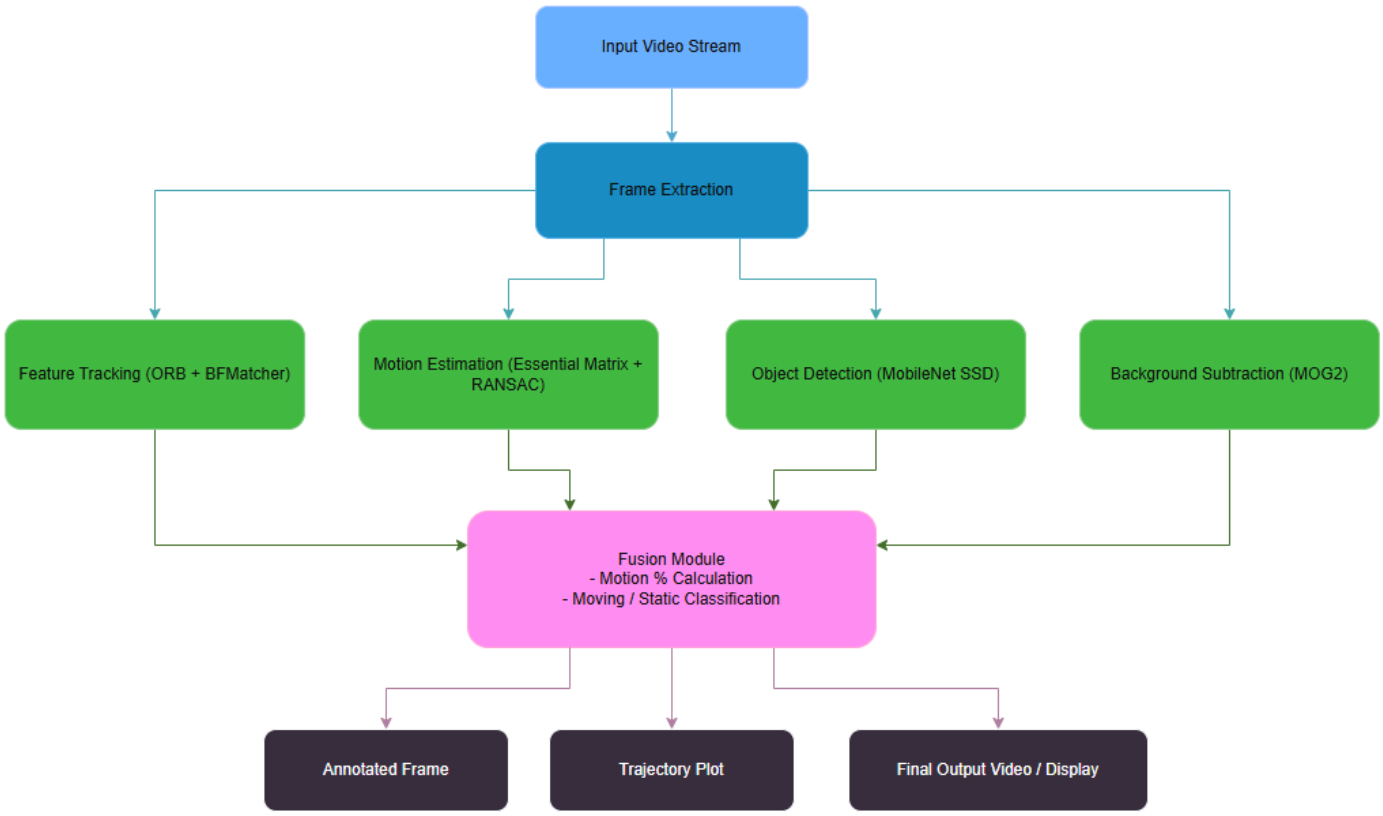

Figure 1 — System Architecture: data flow through the CV motion tracking pipeline

3.2 Feature Detection and Matching

The first branch to run on every frame is the feature tracker, implemented in feature_tracking.py. The tracker uses ORB — Oriented FAST and Rotated BRIEF — to identify up to one thousand interest points in each frame and compute a compact 256-bit binary descriptor for each one. These descriptors encode the local appearance around each keypoint in a form that is invariant to modest rotations and scale changes.

Matching between consecutive frames is handled by a Brute-Force Matcher operating on Hamming distance, which counts the number of bit positions where two binary descriptors differ. A cross-check constraint is applied: a pair of descriptors is accepted as a match only if each is the other's nearest neighbour. This bidirectional test eliminates the majority of spurious correspondences without any additional filtering. The matches that survive are sorted by their

Hamming distance so that the strongest correspondences can be used preferentially in subsequent stages.

The choice of one thousand features per frame represents a deliberate trade-off. Increasing the count produces richer sets of correspondences and, in principle, more robust motion estimates, but it also increases the time spent in descriptor extraction and matching. At one thousand features the tracker runs comfortably within a real-time budget on a standard laptop CPU. The parameter is exposed in the class constructor so that it can be tuned upward on faster hardware.

3.3 Camera Motion Estimation

Once matches have been established, the motion estimator in motion_estimation.py takes the pixel coordinates of each matched pair and attempts to recover the relative pose of the camera between the two frames. The mathematical object at the centre of this computation is the Essential Matrix, a 3×3 matrix that encodes all the information about the rigid-body transformation — rotation and translation — that could have produced the observed pattern of correspondences between a pair of calibrated cameras.

The Essential Matrix is estimated using the five-point algorithm wrapped in RANSAC. RANSAC is necessary because even after cross-check filtering, a fraction of the matches will be geometrically inconsistent — they correspond to moving objects in the scene rather than static background, and they violate the rigid-body assumption that the Essential Matrix depends on. RANSAC handles this by repeatedly hypothesising an Essential Matrix from a minimal random subset of correspondences and counting how many other matches are consistent with that hypothesis. The hypothesis with the most inliers is returned as the estimate.

The rotation R and translation direction t are recovered from the Essential Matrix via OpenCV's recoverPose function. A crucial point about this output is that t is a unit vector: it tells you which direction the camera moved, but not how far. This is the scale ambiguity inherent in monocular vision. No amount of clever mathematics can recover metric scale from a single camera without additional information, because any scene can be uniformly scaled and produce identical images. In this implementation the translation is normalised and multiplied by a configurable scale factor before being accumulated into the trajectory. The resulting path is therefore directional rather than metric, a limitation that is discussed further in Section 5.

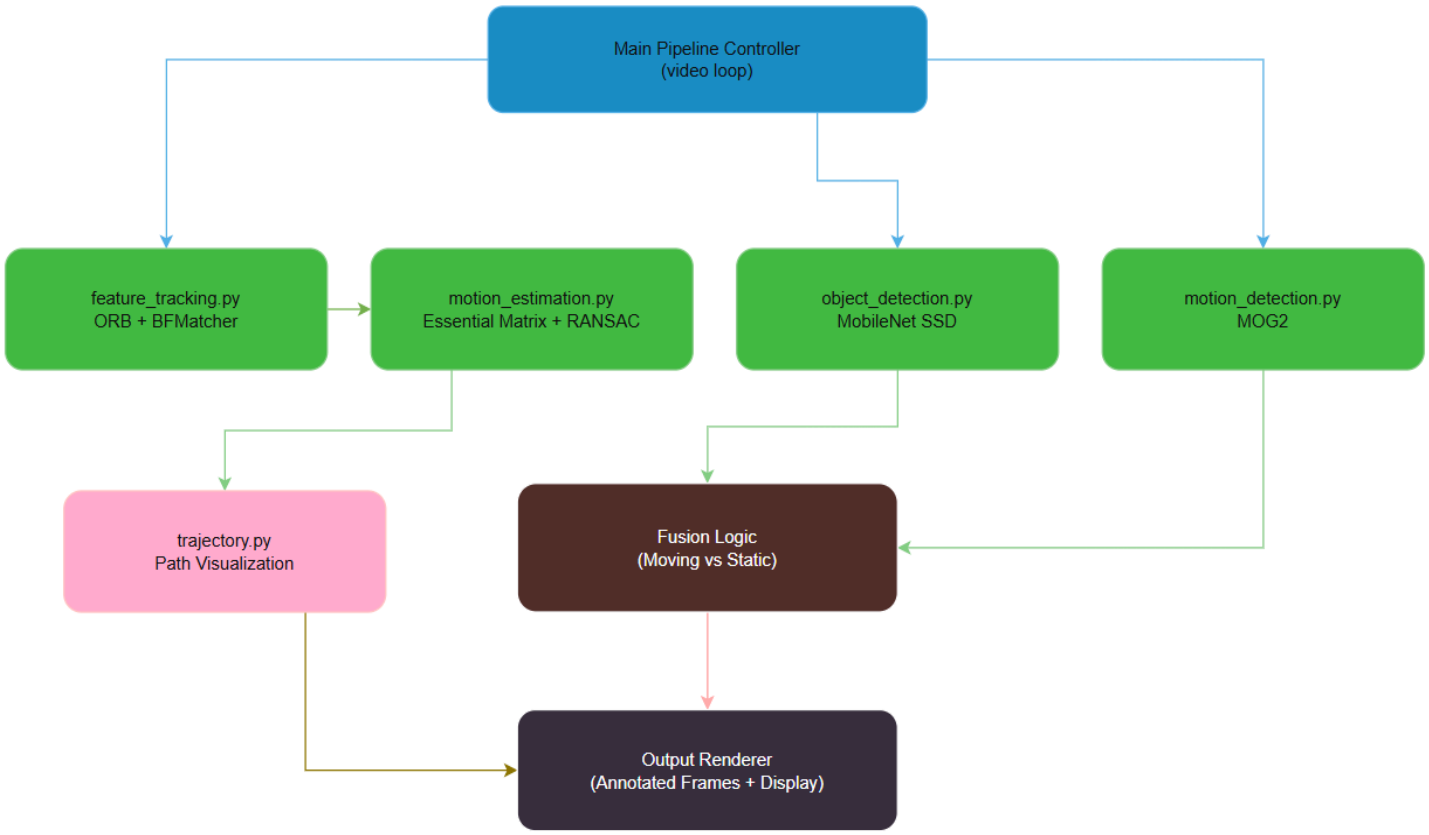

Figure 2 — Module interaction architectural diagram

3.4 Object Detection

Object detection is performed by the ObjectDetector class in object_detection.py, which loads a pre-trained MobileNet Single Shot MultiBox Detector using OpenCV's DNN module. The model was trained on the PASCAL VOC dataset and can recognise twenty object categories including persons, cars, bicycles, buses, motorbikes, and several others that are likely to appear in traffic or outdoor surveillance footage.

The detection process follows the standard SSD pipeline. Each incoming frame is resized to 300×300 pixels, normalised by subtracting the channel means (127.5 across all three channels) and applying a scale factor of approximately 0.0078, and packed into a four-dimensional blob that the network accepts. The network runs a single forward pass and produces a tensor of candidate

detections; those with confidence below 0.5 are discarded, and the bounding boxes of the surviving detections are scaled back to the original frame dimensions.

MobileNet SSD was chosen over heavier alternatives for a practical reason: it runs without any dedicated GPU and requires no additional deep learning framework beyond OpenCV itself. A model like YOLOv8 achieves measurably better detection accuracy on standard benchmarks, but it depends on PyTorch and benefits substantially from GPU acceleration. For a project intended to be runnable on a typical student laptop, MobileNet SSD strikes the right balance.

3.5 Background Subtraction and Motion Classification

The fourth branch, implemented in motion_detection.py, addresses the question of which pixels in the current frame belong to moving objects rather than the static background. The algorithm used is MOG2, which stands for Mixture of Gaussians version 2. MOG2 models the intensity history of each pixel as a mixture of Gaussian distributions, updating the mixture parameters incrementally as new frames arrive. Pixels whose intensity falls outside the expected range of any Gaussian in their mixture are classified as foreground.

Two parameters govern the behaviour of the model in this implementation. The history parameter is set to 500 frames, meaning the model's background estimate integrates information from the past five hundred observations; this makes it slow to react to sustained changes, which reduces false alarms from gradual illumination shifts but means that a newly parked car will remain flagged as foreground for some time before being absorbed into the background. The variance threshold, which controls how far a pixel must deviate from its expected value to be considered foreground, is set to 50. A median blur with a five-pixel kernel is applied to the raw binary mask to suppress the salt-and-pepper noise that arises from camera sensor variation.

The motion label assigned to each detected object is computed by examining the foreground mask inside the object's bounding box. If more than ten percent of the pixels in that region are classified as foreground by MOG2, the object is labelled Moving and its bounding box is drawn in red on the output frame. Otherwise it is labelled Static and drawn in green. The ten percent threshold was chosen empirically: a lower value causes stationary objects on the edge of the background model's adaptation to be incorrectly flagged as moving, while a higher value misses objects that are only partially visible within their bounding box.

3.6 Trajectory Visualisation

The Trajectory class in trajectory.py maintains a running estimate of the camera's two-dimensional path. On each frame where a valid translation vector is recovered, the vector is normalised to unit length, multiplied by a scale factor of ten pixels per unit, and added to the current position. The resulting position is plotted as a small green circle on a 600×600 pixel canvas. The canvas accumulates marks across all frames, producing a top-down trace of the camera's estimated path over the duration of the video. Coordinates are clamped to the canvas boundaries so that erratic motion estimates in problematic frames cannot cause the visualisation to crash.

4. Key Design Decisions

Every implementation involves choices, and the reasoning behind those choices matters as much as the choices themselves. This section documents the most significant decisions made during the development of this system and explains why the alternatives were rejected.

ORB over SIFT and SURF

The two most widely cited alternatives to ORB for real-time feature detection are SIFT and SURF. Both produce floating-point descriptors that are more discriminative than ORB's binary descriptors, particularly under large changes in scale or viewing angle. However, SURF remains commercially restricted in several jurisdictions, and both algorithms are considerably slower than ORB on CPU. Since the entire pipeline must complete within the time budget of a single video frame, the speed advantage of ORB was decisive. On typical traffic footage with modest camera motion between frames, ORB produces enough accurate correspondences to support reliable Essential Matrix estimation.

Cross-Check Matching over Lowe's Ratio Test

Lowe's ratio test, which accepts a match only when the best candidate is significantly closer than the second-best, is the standard filtering strategy for SIFT descriptors. For binary descriptors matched by Hamming distance, cross-check matching tends to be more effective because the discrete nature of Hamming distance makes the ratio less informative. Cross-check

also integrates naturally with OpenCV's BFMatcher interface and avoids the overhead of retrieving two nearest neighbours per query descriptor. The resulting match sets are smaller but cleaner.

Minimum Threshold of Eight Matches for Motion Estimation

The Essential Matrix is determined by a minimum of five point correspondences, but in practice estimating it from exactly five points is numerically fragile. Eight is the minimum count used by the older eight-point algorithm and provides enough geometric constraint to produce a stable estimate in most circumstances. Frames with fewer than eight surviving matches are silently skipped in the motion estimation step rather than attempting to extrapolate or interpolate, which would introduce errors that accumulate over subsequent frames.

MOG2 over Frame Differencing

Simple frame differencing — subtracting consecutive frames and thresholding the result — is a faster and simpler alternative to MOG2. It was rejected because it is highly sensitive to camera shake: any vibration of the camera, even sub-pixel, produces a large foreground response across the entire frame. MOG2, by contrast, maintains a per-pixel model that adapts to slow background changes and is therefore far more robust to the minor camera tremor present in handheld or vehicle-mounted footage. The additional computational cost of MOG2 is modest compared to the object detector, so there was no compelling reason to prefer the simpler method.

Modular Architecture with Dedicated Classes

The decision to encapsulate each processing stage in a dedicated Python class was made at the outset and paid dividends repeatedly during development. When the motion estimator was producing incorrect results due to an error in how matches were indexed, the bug was isolated to a single file without any need to inspect the object detector or background subtractor. When the trajectory scale needed tuning, only the Trajectory constructor required modification. This kind of localised change is only possible when the concerns of each stage are genuinely separated from those of the others.

5. Challenges Encountered

Building a system that integrates four distinct computer vision techniques and runs in real time on CPU raised several non-trivial challenges. They are documented here not merely as a record of obstacles overcome, but because the way a challenge is resolved reveals as much about a design as the initial architecture does.

Scale Ambiguity in Monocular Odometry

This was the most fundamental limitation of the approach and the one that was recognised early enough to be addressed through design rather than engineering. Monocular visual odometry cannot recover metric scale. The translation vector recovered by recoverPose is always of unit Euclidean norm — it encodes direction but not magnitude. There is no mathematical workaround for this without additional information, such as a known object size in the scene, a second camera, or an inertial measurement unit. The chosen response was to normalise the translation, multiply by a fixed scale factor, and clearly document that the resulting trajectory is directional rather than metric. Users who require accurate scale are pointed toward camera calibration and, ultimately, toward a stereo or IMU-augmented setup.

Camera Intrinsic Matrix Mismatch

The Essential Matrix computation requires the camera's intrinsic matrix K, which encodes the focal length and principal point. The default values used in this implementation come from the KITTI autonomous driving dataset. These values are reasonable for a forward-facing automotive camera but will produce geometrically incorrect motion estimates for any other camera. This was addressed by logging a prominent warning whenever the default values are used, documenting the calibration procedure in both the code and the README, and making the matrix a first-class parameter of the MotionEstimator constructor so that calibrated values can be substituted without any code changes.

Degenerate Frames in Feature Matching

Not every frame in a real video contains enough texture to support reliable feature detection. Frames that are heavily blurred, overexposed, or dominated by large uniform surfaces yield very few ORB keypoints, and the number of surviving cross-check matches may fall below the threshold required for Essential Matrix estimation. The system handles these frames gracefully

by checking the match count before attempting estimation and silently skipping the motion estimation step when the count is insufficient. This prevents a single difficult frame from crashing the pipeline or introducing a large erroneous jump in the trajectory.

Bounding Box Coordinate Clamping

Object detection operates on a resized version of the frame (300×300 pixels for the network input) and then projects bounding box coordinates back to the original resolution. This projection occasionally produces coordinates that fall outside the valid pixel range of the frame, particularly for objects detected near the edges. Accessing a NumPy array slice with out-of-range indices raises no exception in Python but silently returns an empty array, which would cause a division-by-zero error in the foreground fraction calculation. The solution is to explicitly clamp all four bounding box coordinates to the frame dimensions before the ROI is extracted.

MOG2 Adaptation Lag

When the video begins, the background model has no history and effectively classifies everything as foreground. This produces a period of several seconds at the start of each video during which almost all detected objects are incorrectly labelled as moving. The adaptation lag can be reduced by pre-warming the model on the first few frames before displaying any output, but this was not implemented in the current version. Users processing footage where the camera begins moving immediately should be aware of this initialisation artefact.

6. Reflections and Learning Outcomes

The experience of building this system from scratch produced insights that reading about computer vision does not. Several of the most valuable ones are recorded here.

The Distance Between Understanding and Implementation

Before starting this project, I had a reasonable theoretical understanding of how the Essential Matrix is derived and what it encodes. What I had not appreciated was how many things can go wrong between the textbook derivation and a working implementation. The Essential Matrix assumes that all matched point pairs are consistent with a single rigid-body transformation, which is violated by moving objects in the scene. It assumes calibrated cameras, which means the

intrinsic matrix must be correct. It assumes sufficient baseline between the two views, which means nearly pure rotation produces a degenerate solution. None of these caveats are prominently featured in introductory treatments of the topic, and discovering them empirically — through incorrect trajectories, crashed functions, and mysteriously null outputs — was a far more lasting form of education than reading about them would have been.

Camera Calibration as a First-Class Engineering Concern

The KITTI default intrinsic matrix used in this implementation is, for most cameras, completely wrong. When the implementation first ran on the test video without a calibrated K matrix, the trajectory bore no resemblance to the actual path of the camera. Switching to values that better approximated the test camera's focal length produced a trajectory that was at least directionally plausible. This experience made concrete something that is easy to gloss over in coursework: geometric vision is entirely dependent on an accurate model of the camera's optics, and that model must be measured, not assumed.

Classical and Learned Methods as Complements

There is a narrative in the computer vision community — sometimes stated explicitly, more often implied — that deep learning is gradually making classical geometric methods obsolete. This project argues otherwise, at least at the level of the problems addressed here. The neural network (MobileNet SSD) is essential for answering the question of what an object is; no amount of geometric analysis of pixel correspondences will tell you that a rectangular blob is a car rather than a bus. But the Essential Matrix is essential for answering the question of where the camera is going; a neural network trained on static images has no particular reason to produce calibrated pose estimates. The two families of technique address genuinely different aspects of the problem, and the most effective systems use both.

The Real-Time Constraint as a Design Discipline

Implementing a system that must process video in real time imposes a discipline that offline batch processing does not. Every component must be efficient enough that the total per-frame budget is not exceeded, which forces explicit reasoning about the cost of each design choice. The decision to use one thousand rather than two thousand ORB features, to use MobileNet rather than a heavier detector, to skip motion estimation on frames with too few matches rather

than attempting a fallback — all of these were shaped by the real-time requirement. Working within that constraint turned out to be one of the most practically valuable aspects of the project.

The Value of Defensive Programming in Pipelines

A video pipeline processes thousands of frames, and unusual inputs are not the exception — they are inevitable. The bounding box clamping bug, the degenerate-frame crash, the zero-norm translation edge case: each of these manifested only on specific frames that happened to trigger a rare condition. A system that is not explicitly defended against these conditions will fail in production, and the failures will be difficult to reproduce and diagnose. The habit of asking, for every function, what happens when the input is empty, zero, or out-of-range — and then handling those cases explicitly — is one that transfers directly to any engineering context.

7. Summary

This project set out to build a computer vision pipeline that could simultaneously detect and track features, estimate camera motion, identify objects, and classify those objects as moving or stationary — all in real time, all from a standard video file, all on hardware that a student or developer is likely to have available. The system that resulted achieves these goals within the constraints of monocular vision and CPU-only processing, and does so through a modular architecture that makes individual components straightforward to test, tune, or replace.

The limitations of the approach are real and worth restating plainly. Metric scale cannot be recovered from a single camera. The default camera intrinsic values are incorrect for any camera other than the one used to collect the KITTI dataset. MobileNet SSD, while fast and convenient, produces noticeably more false positives and misses more difficult detections than current state-of-the-art models. The MOG2 model requires several seconds to initialise at the start of each video.

These limitations, however, do not diminish the value of what the system demonstrates. The pipeline shows how geometric vision, statistical background modelling, and deep learning-based detection can be combined in a coherent and practically useful way. It also shows, perhaps more importantly, that building systems which work is harder and more instructive than building systems which are theoretically sound. The gap between a correct algorithm and a robust

implementation is where the real engineering happens, and closing that gap is what this project was ultimately about.

Appendix — Pipeline Components at a Glance

Module	Purpose	Algorithm	Output
feature_tracking.py	Detect & match keypoints	ORB + BFMatcher (Hamming)	kp[], des[], DMatch[]
motion_estimation.py	Recover camera pose	Essential Matrix + RANSAC	R (3×3), t (3×1)
trajectory.py	Visualise camera path	Cumulative translation	600×600 canvas
object_detection.py	Detect & classify objects	MobileNet SSD (Caffe)	label, conf, bbox
motion_detection.py	Segment moving pixels	MOG2 background subtraction	binary fgmask